

Session 10 — Project Guide: Component Architecture

This guide explains, in depth, what changed in TaskFlow during Session 10 and — more importantly — WHY it changed. If you forgot most of class, you should be able to read this alone and rebuild your understanding.

A promise up front: nothing the app DOES is different after today. Every button, every filter, every fetch behaves exactly as it did at the end of Session 09. What changed is WHERE the code that does those things lives, and HOW some of it reads its own inputs.

1. Where today's session starts

Session 10 starts from Session 09's End: real fetched `tasks` and `users`, independent loading/error/Retry for each, filtering, search, person selection, stats, and Add/Toggle/Delete/Edit — all already working. `App.tsx` had grown to over 370 lines doing every one of those jobs by itself: holding the type definitions, holding the fetch functions, holding every piece of state, holding every handler, AND holding almost the entire page's JSX.

2. Why one giant file becomes a problem

Think of a familiar idea first: a junk drawer. One drawer that holds batteries, tape, spare keys, and old receipts all together works fine when the drawer is small. As it fills up, finding anything gets slower, and it becomes easy to disturb something you weren't looking for while digging for something else. `App.tsx` was that drawer. A small change to how the task list renders shouldn't require scrolling past unrelated fetch logic and unrelated stats markup to find the right spot — and shouldn't risk accidentally breaking something nearby while you're in there.

The fix is not "delete code" — every line still exists and still runs. The fix is giving each concern (data shapes, network requests, one section of the page) its own small, named file.

3. Object/props destructuring — the first new idea

Every component so far has taken one `props` object and reached into it repeatedly:

```
function StatCard(props: StatCardProps) {
  return (
    <div className="stat-card">
      <p className="stat-label">{props.label}</p>
      <p className="stat-value">{props.value}</p>
```

```
    </div>
  );
}
```

That works. But notice you have to read the WHOLE function body to discover that `StatCard` only actually cares about two fields, `label` and `value`. Destructuring unpacks named fields directly in the parameter list:

```
function StatCard({ label, value }: StatCardProps) {
  return (
    <div className="stat-card">
      <p className="stat-label">{label}</p>
      <p className="stat-value">{value}</p>
    </div>
  );
}
```

Line by line:

- `{ label, value }` — this is NOT creating a new object. It is a DESTRUCTURING PATTERN: "take the one object being passed in, and pull `label` and `value` out of it into two plain variables with those same names."
- `: StatCardProps` — the type annotation is unchanged in meaning. It still describes the shape of the ONE incoming argument — it just now sits after the destructuring pattern instead of after a variable named `props`.
- Inside the function body, `label` and `value` are now plain variables, exactly as if you had written `const label = props.label;` and `const value = props.value;` yourself, except React (and TypeScript) do this unpacking for you, automatically, every time the component is called.

This is not a React feature — it is plain JavaScript/TypeScript. You could destructure the parameter of any ordinary function the same way, whether or not it returns JSX.

WRONG / RIGHT:

```
// WRONG — this repeats the field names in the type position, which is not what
// a type annotation is for. TypeScript needs real type names here.
function StatCard({ label, value }: { label, value }) { ... }

// RIGHT — the type annotation names an actual interface describing the shape.
function StatCard({ label, value }: StatCardProps) { ... }
```

4. React Fragment — the second new idea

Here is a rule you have followed since your very first component, maybe without ever questioning it: a component's `return` produces exactly ONE value. Every component so far solved "I have several sibling pieces of markup to return" by wrapping them all in one `<div>`.

Today, `Controls` needs to return three sibling pieces: the filter buttons, the search box, and the "X of Y tasks shown" line. Wrapping them in a `<div>` would work — but that `<div>` would have no CSS class, no styling purpose, and no reason to exist beyond satisfying the "return one thing" rule. It would sit in the real page's HTML forever, doing nothing useful.

```
function Controls({ ... }: ControlsProps) {
  return (
    <>
      <FilterButtons currentFilter={currentFilter} onChange={onFilterChange} />
      <SearchInput value={searchText} onChange={onSearchChange} />
      <p className="visible-count">
        {visibleCount} of {totalCount} tasks shown
      </p>
    </>
  );
}
```

`<>` and `</>` are a Fragment — a way to group multiple sibling elements into one value WITHOUT adding any real element to the page. Opening a browser's element inspector before and after this change would show the exact same three pieces of markup, just with one less empty wrapping `<div>` in between them.

Common mistake: opening with `<>` and closing with `</div>` (or the reverse). The two must match — `<>` closes with `</>`, nothing else. TypeScript/your build tool will report a clear syntax error if they don't match, which is a good, safe way to notice the mistake.

When NOT to use a Fragment: if a wrapper genuinely needs a CSS class, an `onClick`, or any other attribute, it needs to be a real element (like a `<div>` or `<section>`), not a Fragment — a Fragment accepts essentially no attributes, because it doesn't produce a real DOM node to attach them to.

5. `src/types.ts` — organizing, not learning something new

You already know what an `interface` and a `type` alias are — you used both in earlier sessions. Today's change is purely about WHERE they live:

```

export interface Task {
  id: number;
  userId: number;
  title: string;
  completed: boolean;
}

export interface User {
  id: number;
  name: string;
}

export type FilterStatus = "all" | "completed" | "pending";

```

These three declarations used to sit directly inside `App.tsx`. Now every file that needs to know what a `Task` or `User` looks like — and after today, that's a dozen different component files — imports it from one shared place:

```

import type { Task, User, FilterStatus } from "../types";

```

Think of it like a shared vocabulary list at the front of a textbook: every chapter can refer back to it, instead of each chapter re-explaining the same words in its own way.

6. `src/api.ts` — the same idea, applied to fetching

`TASKS_URL`, `USERS_URL`, `fetchTasks`, and `fetchUsers` moved out of `App.tsx` into `src/api.ts`. Nothing about HOW they fetch changed — same URLs, same `async / await`, same `.json()` parsing you already learned in Session 09:

```

import type { Task, User } from "../types";

const TASKS_URL = "https://jsonplaceholder.typicode.com/todos?_limit=50";
const USERS_URL = "https://jsonplaceholder.typicode.com/users";

export async function fetchTasks(): Promise<Task[]> {
  const response = await fetch(TASKS_URL);
  return (await response.json()) as Task[];
}

export async function fetchUsers(): Promise<User[]> {

```

```
const response = await fetch(USERS_URL);
return (await response.json()) as User[];
}
```

`App.tsx` now just imports both functions and calls them exactly as before:

```
import { fetchTasks, fetchUsers } from "../api";
```

7. Who owns state, and who just displays it — the central architecture question

This is the most important idea in today's session, more important than any single piece of syntax.

The rule: state lives in the CLOSEST component that is a common ancestor of every component that needs it. `App` still owns every single `useState` call from Session 09 — `tasks`, `users`, `isLoadingTasks`, `hasTaskError`, `hasUsersError`, `currentFilter`, `searchText`, `selectedUserId`. Not one of today's new components introduces its own state for something `App` already tracks.

Instead, every new component receives what it needs to DISPLAY through props, and reports what happens through callback props — the same "state down, actions up" pattern you already used for individual buttons in earlier sessions, now applied to whole sections of the page.

```

App (owns ALL the state)
|
| — StatsBar (total, completed, pending) — receives numbers,
displays them
|
| — Controls (currentFilter, searchText, ...) — receives values + 2
callbacks
|   | — FilterButtons
|   | — SearchInput
|
| — PeopleSummary (users, tasks, selectedUserId, ...) — receives data
+ 2 callbacks
|   | — PersonButton (one per person)
|
| — AddTaskForm (users, selectedUserId, onAddTask)
|
```

```
└─ TaskList (tasks, getOwnerName, onToggle, onDelete, onSaveEdit)
    └─ TaskItem (one per task)
        └─ Badge
```

Ask yourself, for any new component: "What does this need to KNOW to render itself?" (its props flowing down) and "What does it need to REPORT back up when something happens?" (its callback props). Answering those two questions honestly is how you decide a component's exact prop list — you don't need to guess or copy another component's shape.

8. Tracing one click through the whole architecture

Let's trace exactly what happens when a user clicks a person's name button, from the very first event to the screen updating — this is the mental model to have ready any time something doesn't work as expected.

1. The click happens on a `<button>` rendered inside `PersonButton`.
2. `PersonButton` doesn't know or care what "selecting a person" means — it just calls whatever function was handed to it as `onClick`.
3. That `onClick` was set by `PeopleSummary`, as `() => onSelectPerson(entry.user.id)` — so calling it calls `PeopleSummary`'s own `onSelectPerson` prop, with this one person's real id.
4. `PeopleSummary` didn't invent `onSelectPerson` either — `App` passed its own `handleSelectPerson` function down as that exact prop.
5. `handleSelectPerson` runs inside `App`, and calls `setSelectedUserId(userId)`.
6. React sees state change, and re-renders `App` — which re-renders everything below it with the NEW `selectedUserId` value, including a new `visibleTasks` calculation (filtered to that person) and a re-rendered `PeopleSummary` (now highlighting the newly active `PersonButton`).

Notice: the actual STATE change (`setSelectedUserId`) only ever happens in ONE place, `App`. Every component in between is just relaying a function call, unchanged, from parent to child.

9. Debugging a broken prop or callback chain

When something that "should" work doesn't — a click does nothing, or a value doesn't show up — walk the same chain from Section 8, but backwards, from where the screen is wrong toward where the state lives:

1. **Is the value actually reaching the component that displays it?** Temporarily add a `console.log` of the prop right where it's used. If it's `undefined` or wrong there, the problem is further UP the tree — check what the parent passed in.

2. **Is the prop NAME the same on both sides?** TypeScript will usually catch a straight-up typo (`onChange` vs `onFilterChange`) with a compiler error — read that error message, it names the exact mismatch.
3. **Is the callback actually being CALLED?** A button whose `onClick` was written as `onClick={onSelectPerson}` instead of `onClick={() => onSelectPerson(entry.user.id)}` will call `onSelectPerson` with a click EVENT as its argument, not the person's id — a very easy mistake once several layers of callbacks are involved. Check every arrow-function wrapper is present where an argument needs to be supplied.
4. **Did the state update happen in the component that actually owns that state?** If you find yourself wanting to add a `useState` inside a new child component "just to make this work," stop and ask whether that value is already owned by a parent — duplicating state in two places is a common source of the UI "getting out of sync with itself."

10. `PeopleSummary` + `PersonButton` — fixing a real duplication

Before today, the people list rendered TWICE: once as plain `PersonSummary` paragraphs (just names and counts, no click behavior), and again as a separate row of clickable named buttons, built later, in a different session, for a different immediate purpose. Nobody had gone back to unify the two. `PeopleSummary` now owns both jobs — displaying people AND letting the user click one — as a single component. It calculates its own `peopleWithCounts` internally, rather than receiving it pre-computed, because that calculation is a detail only `PeopleSummary` itself needs.

11. `TaskItem` now takes one `task` object, not four separate fields

Before today, `App` computed `statusText` and `statusClass` from `task.completed` every single render, then passed FOUR separate props (`id` , `title` , `statusText` , `statusClass`) into `TaskItem` . Now that a shared `Task` type exists, `TaskItem` simply receives the whole `task` object:

```
<TaskItem
  key={task.id}
  task={task}
  ownerName={getOwnerName(task.userId)}
  onToggle={onToggle}
  onDelete={onDelete}
  onSaveEdit={onSaveEdit}
/>
```

and derives what it needs to display directly from `task.completed` , using a small new `Badge` component for the status pill:

```
<Badge status={task.completed ? "completed" : "pending"} />
```

This removes a redundant derivation step, and removes any chance of the four separate pieces ever silently getting out of sync with each other.

12. What did NOT change

`TaskItem` 's local editing behavior — `isEditing`, the draft title, the empty/200-character validation — is exactly the same logic as Session 08/09, just living inside a destructured component now instead of a flat-props one. `AddTaskForm` 's safe-default-owner logic from Session 09's teamwork is unchanged in behavior. Every state variable in `App` still exists, with the same name and the same meaning it always had.

Glossary

- **Destructuring** — unpacking named fields out of an object directly, instead of accessing them through a variable one field at a time.
- **Fragment** (`<>...</>`) — a way to group multiple sibling JSX elements into one return value without adding a real element to the page.
- **Component tree** — the parent/child structure formed by which components render which other components.
- **State ownership** — the decision of which single component's `useState` is the source of truth for a given value.
- **Prop drilling** — passing a prop down through one or more components that don't use it themselves, only to reach a component further down that does. (Not a problem you'll hit badly at this app's size — worth knowing the name, since it explains why very deep component trees sometimes reach for more advanced tools later.)

Common mistakes

- Destructuring a prop but still writing `props.x` on one leftover line — TypeScript will report `props` as not defined once the parameter is fully destructured; read that error rather than guessing.
- Mismatched Fragment tags (`<>` closed with `</div>` , or vice versa).
- Passing a pre-computed derived value into a component that should calculate it itself (or the reverse) — before writing a prop, ask "does the CHILD need to know this, or does it need to COMPUTE this itself from something simpler it's already given?"
- Introducing a new `useState` inside a child component for a value the parent already owns and passes down — this creates two separate "sources of truth" for the same idea, which can

silently drift out of sync.

- Assuming a refactor session is a good moment to also rename or "clean up" something unrelated — today's changes are deliberately scoped to exactly what's described above.

Review questions

1. What problem does destructuring solve, and what problem does it NOT solve?
2. Why can't `Controls` just use a `<div>` instead of a `Fragment`?
3. Why does `PeopleSummary` calculate `peopleWithCounts` itself instead of receiving it as a prop?
4. Trace what happens, step by step, when a user clicks "Mark Completed" on a task.
5. Why does `App` still hold `selectedUserId`, even though only `PeopleSummary` / `PersonButton` visibly react to it being clicked?

Answers

1. It solves HOW a component reads its own props (unpacking named fields instead of writing `props.x` repeatedly). It does not change what data the component receives, or how a parent calls it — a destructured component is called exactly the same way as before.
2. A `Fragment` groups `Controls`'s three sibling pieces without adding a real, purposeless element to the page. A `<div>` would work, but it would have no CSS class and no reason to exist beyond satisfying the "return one thing" rule — an empty wrapper sitting in the DOM for no reason.
3. Because it's a detail only `PeopleSummary` itself needs in order to render — `App` only needs to hand it the raw `users` and `tasks` arrays it already owns; there's no reason for `App` to also own the derived "who has how many tasks" calculation when nothing else in `App` uses it.
4. The click fires on a `<button>` inside `TaskItem`. `TaskItem` calls its own `onToggle` prop with `task.id`. That `onToggle` prop is `App`'s `handleToggle` function, passed down through `TaskList` unchanged. `handleToggle` runs in `App`, builds a new `tasks` array with that one task's `completed` flipped, and calls `setTasks`. React re-renders `App` and everything below it with the updated array.
5. Because `selectedUserId` also affects `visibleTasks`, which `TaskList` needs — `App` is the closest common ancestor of BOTH `PeopleSummary` (which displays/reacts to it) and `TaskList` (which is filtered by it). If `PeopleSummary` owned it alone, there would be no way for that value to also reach `TaskList`.

What you should be able to explain now

- Why `App.tsx` being one giant file eventually becomes a real problem, not just an aesthetic one.
- How to destructure a component's props, and why doing so doesn't change what the component does.
- What a Fragment is for, and when a real wrapping element is still the right choice instead.
- How to decide which component should own a given piece of state.
- How to trace a click (or any user interaction) all the way from the DOM event to the state update and back down to the re-render, through however many components sit in between.
- Why today's refactor should not have changed a single thing about what the app does when you use it.

Not yet

The following ideas have NOT been taught yet, even though some of today's code sits right next to where they'll eventually go. Their NAMES may be worth knowing; their syntax is not shown here on purpose:

- A Reset button that clears all filters/search/selection at once.
- Deleting a task with a confirmation step (`window.confirm` or similar).
- Editing a task's owner, or combining title and owner into one edit action.
- Using the keyboard (Enter to save, Escape to cancel) while editing a task.
- Anything that changes what `AddTaskForm` shows or does based on whether people have finished loading, beyond the safe-default-owner guard you already know from Session 09.

All of the above belong to Session 11.